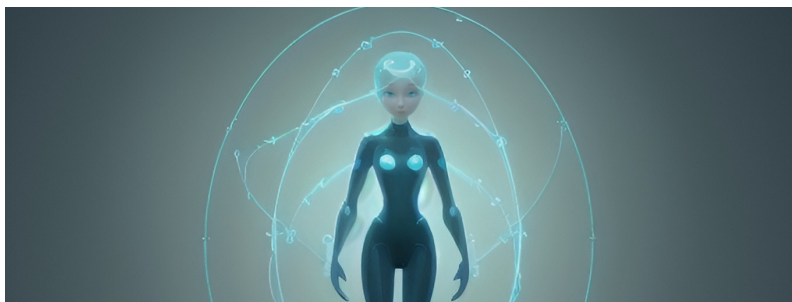


Artificial Intelligence for Digital Characters

Solution Exercise 5

Hand-out Date: 27 May 2026



Goals

- Understand the fundamentals of autonomous agents and their knowledge representation and decision-making processes.
- Understand approaches for implementing decision-making processes in agents using dialogue trees and knowledge graph embeddings.
- Understand the application of reinforcement learning in non-differentiable simulation systems and robotic control tasks.
- Learn to formulate reinforcement learning problems by properly defining states, actions, rewards, and policies for complex motion tracking tasks.
- Develop practical knowledge of reinforcement learning training methodologies including data collection strategies, policy updating algorithms, and implementation considerations.

Resources

The lecture slides, exercise slides, and Unity template are accessible via Moodle.

Tasks

1. Multiple Choice Questions

Please mark the correct answer for the multiple-choice questions below.

- a) False – Proactiveness in autonomous agents refers to their ability to plan for the future to take advantage of opportunities and avoid problems. It is reactivity, not proactiveness, that is characterized by perceiving and responding to immediate changes in the environment in a timely fashion.
- b) False – Covering new topics or scenarios with dialogue trees requires manual new design, making them difficult to scale efficiently.
- c) True.
- d) True.
- e) False - the transition model does not necessarily need to be differentiable.
- f) True.
- g) False - the optimal policy can maximize the expected return, rather than each individual reward in the future steps
- h) True.

2. Knowledge Graphs

a)

1. *Relevant Entities:*

Renaissance , 15th century , European cultural movement , 16th century , Middle Ages , Republic of Florence , Italy , Europe , humanism , classical Greek philosophy

Selection Criteria:

The goal is to create a KG relevant for a museum guide agent. Therefore, the criteria focused on selecting entities that represent:

Core Topic: Renaissance is the central subject, which is a European cultural movement

Temporal Context: 15th century and 16th century define the period; Middle Ages provides the preceding context. These are essential for historical framing often sought by museum visitors.

Geographic Context: Republic of Florence (origin), Italy, and Europe (spread) are crucial locations relevant to understanding the movement's scope and likely relevant to museum exhibits (e.g., Florentine art, Italian Renaissance).

Key Intellectual Concepts: humanism is identified as the core intellectual basis, and classical Greek philosophy is a key source mentioned. These concepts are fundamental to understanding the Renaissance's character.

2. *Relationships based on text for selected entities:*

Renaissance is_a European cultural movement

Renaissance spans 15th century

Renaissance spans 16th century

Renaissance marked_transition_from Middle Ages

Renaissance began_in Republic of Florence

Renaissance spread_to Italy

Renaissance spread_to Europe

Renaissance intellectual_foundation_based_on humanism

humanism derived_from classical Greek philosophy

b) Using the knowledge graph you created in a), first classify the type of query each question represents, then write SPARQL queries to retrieve the relevant information:

1. *From which historical period is the Renaissance transitioned?*

Query Type: One-hop Query

SPARQL Query:

```
PREFIX ren: <http://example.org/renaissance#>
SELECT ?period
WHERE {
    ren:Renaissance ren:marked_transition_from ?period .
}
```

2. What ancient philosophical tradition formed the foundation of the Renaissance's intellectual basis?

Query Type: Path Query

SPARQL Query:

```
PREFIX ren: <http://example.org/renaissance#>

SELECT ?philosophical_tradition
WHERE {
    ren:Renaissance ren:intellectual_foundation_based_on ?foundation .
    ?foundation ren:derived_from ?philosophical_tradition .
}
```

c) When the knowledge graph is large and incomplete, knowledge graph embeddings (KGE) can be used for information retrieval.

1. TransE embeds entities and relationships into a continuous vector space where semantically similar entities appear close together. To find the classical Greek philosopher most similar to Protagoras:

Step 1: Ensure all entities, including Protagoras and other Greek philosophers, are embedded in the same vector space using TransE.

Step 2: Calculate similarity scores between Protagoras's embedding vector and all other philosopher entities using cosine similarity:

$$\text{similarity} = \frac{\vec{v}_{\text{Protagoras}} \cdot \vec{v}_{\text{Philosopher}}}{\|\vec{v}_{\text{Protagoras}}\| \cdot \|\vec{v}_{\text{Philosopher}}\|} \quad (1)$$

Step 3: Rank all philosophers by similarity score and retrieve the highest-scoring entity.

2. “Find cultural movements that were based on ideas derived from classical Greek philosophy AND spread across Europe after the Middle Ages.”

Query type: Conjunctive Query

Answer the query using box embeddings as described in the Query2box framework:

Step 1: Initial Box Construction

- Create Box_1 representing “classical Greek philosophy” as a point (zero-volume box)
- Create Box_2 representing “Europe” as a point
- Create Box_3 representing “Middle Ages” as a point

Step 2: Projection Operations

- Project Box_1 with *derived_from* relation: $\text{Box}_4 = P(\text{Box}_1, \text{derived_from}^{-1})$

This expands to include entities derived from classical Greek philosophy

- Project Box_4 with *intellectual_foundation_based_on* relation: $\text{Box}_5 = P(\text{Box}_4, \text{intellectual_foundation_based_on}^{-1})$

This expands to movements based on foundations derived from Greek philosophy

- Project Box_2 with *spread_to* relation: $\text{Box}_6 = P(\text{Box}_2, \text{spread_to}^{-1})$

This expands to include movements that spread to Europe

- Project Box_3 with *marked_transition_from* relation: $\text{Box}_7 = P(\text{Box}_3, \text{marked_transition_from}^{-1})$

This expands to movements that came after the Middle Ages

Step 3: Intersection Operation

- Calculate the final box: $\text{Box}_{final} = I(\text{Box}_5, \text{Box}_6, \text{Box}_7)$
- The center of this intersection box is the weighted sum of input box centers
- The boundary is determined by the geometric intersection

Step 4: Result Retrieval

- Identify entities contained within Box_{final}
- The “Renaissance” would be the entity within this box, satisfying all conditions

d) List at least 2 practical situations where knowledge graphs can effectively support LLM agents.

Multi-Agent Systems

- Smart home coordination: Agents share KGs to optimize energy, security, and appliance management.

- Healthcare collaboration: Multiple agents extract and refine medical data with minimal human intervention.

Semantic Search and Recommendation

- Content platforms use KGs linking genres, creators, and user ratings to deliver personalized recommendations.

- E-commerce systems leverage KGs to understand user preferences by encoding purchase history and product attributes.

Ethical AI and Bias Mitigation

- HR systems validate job listings against diversity KGs to eliminate gendered language.

- Financial applications use KGs to explain AI decisions (e.g., “Loan denied due to income $\leq$ threshold”), enhancing transparency.

3. Reinforcement Learning

a) RL formulation

1. Why to use reinforcement learning in this task, instead of supervised learning or unsupervised learning?

Reinforcement learning is the appropriate choice for training a humanoid robot to track human walking motion while maintaining balance because the simulation system is typically not differentiable. This creates a fundamental challenge that makes supervised learning approaches ineffective.

In this task, the difference between the desired human motion and the robot's motion can not be minimized through direct backpropagation. The physics simulation creates a complex, non-differentiable barrier between the control inputs (motor torques) and the resulting motion outcomes.

2. Specifically, define the agent, the world, the action space, the reward, the state, and the policy function, respectively.

Agent: The humanoid robot.

World: The simulation environment.

Action space: The set of all possible motor torques that can be applied to the robot's joints.

Reward: Composed of two parts: (1) a term encouraging similarity between the robot's motion and the reference human motion, and (2) a penalty for falling.

State: Includes (1) the current configuration of the robot (e.g., joint angles, velocities), and (2) the discrepancy between the robot's motion and the target human motion.

Policy function: A function that maps the current state to an action (motor torques).

b) Describe your plan to train the policy neural network.

1. How to collect data?

To collect data for training the policy network, we follow a two-step approach: First, we initialize the policy neural network with random weights or pretrained parameters from similar tasks if available. This provides a starting point for exploration in the environment.

Second, we perform multiple simulation episodes (either sequentially or in parallel) using this policy to collect RL trajectories. Each trajectory contains state-action-reward-next state tuples that capture the agent's interactions with

the environment. As training progresses, we continuously update the policy network and collect new trajectories using the improved policy, gradually refining the humanoid's ability to track the reference motion while maintaining balance.

2. How to update the policy network with the Actor-critic (A2C) paradigm?

In the Actor-critic paradigm, we maintain two neural networks: the policy network (actor) that outputs actions and a value network (critic) that estimates state values. To update these networks:

First, we implement an algorithm such as PPO (Proximal Policy Optimization) or TRPO (Trust Region Policy Optimization) to update the policy network. Using the collected trajectories, we compute the expected returns, the estimated state values from our critic network, and the advantage values (the difference between actual returns and estimated values).

Second, we update the policy network by optimizing the clipped PPO objective, which encourages improvements while preventing excessive policy changes. Simultaneously, we update the value network by minimizing a separate loss function (typically mean squared error) between predicted state values and the actual discounted returns observed in the trajectories.

3. Why should we terminate the simulation when the robot has fallen to the ground?

When the robot falls to the ground, we should terminate the simulation primarily to improve data collection efficiency. Once fallen, the reward values become consistently low and provide minimal additional learning signal for the agent. Continuing the simulation in this failed state wastes computational resources.

Moreover, if we collect too many trajectories dominated by fallen states with poor rewards, the policy network training will be biased toward these failure cases. This imbalance makes it difficult for the agent to learn the correct balancing and motion tracking behaviors. Early termination allows us to reset and collect more diverse trajectories with potentially successful attempts, providing more valuable training data to improve the policy.

4. Can we use deep Q-learning (DQN) to achieve our goal?

Yes, we could use DQN, but we would first need to discretize the action space, since DQN only works with discrete action spaces. The motor torque commands for the humanoid robot naturally exist in a continuous space, which isn't directly compatible with DQN's approach of estimating Q-values for each discrete action.

This discretization would involve dividing the continuous range of possible torque values into a finite set of discrete options, which introduces a significant limitation. Coarse discretization reduces control precision, while fine

discretization exponentially increases the action space size, making learning inefficient. Therefore, policy gradient methods like PPO or SAC that naturally handle continuous action spaces are generally more appropriate for humanoid control tasks.